

To suppress the space between items, again, don't print this way. Instead, build up an output string using the string concatenation and formatting tools covered in [Chapter 7](#), and print the string all at once:

```
>>> print x + y
ab
>>> print '%s...%s' % (x, y)
a...b
```

As you can see, apart from their special syntax for usage modes, 2.6 `print` statements are roughly as simple to use as 3.0's function. The next section uncovers the way that files are specified in 2.6 `prints`.

Print Stream Redirection

In both Python 3.0 and 2.6, printing sends text to the standard output stream by default. However, it's often useful to send it elsewhere—to a text file, for example, to save results for later use or testing purposes. Although such redirection can be accomplished in system shells outside Python itself, it turns out to be just as easy to redirect a script's streams from within the script.

The Python “hello world” program

Let's start off with the usual (and largely pointless) language benchmark—the “hello world” program. To print a “hello world” message in Python, simply print the string per your version's print operation:

```
>>> print('hello world')           # Print a string object in 3.0
hello world

>>> print 'hello world'           # Print a string object in 2.6
hello world
```

Because expression results are echoed on the interactive command line, you often don't even need to use a `print` statement there—simply type the expressions you'd like to have printed, and their results are echoed back:

```
>>> 'hello world'                 # Interactive echoes
'hello world'
```

This code isn't exactly an earth-shattering piece of software mastery, but it serves to illustrate printing behavior. Really, the `print` operation is just an ergonomic feature of Python—it provides a simple interface to the `sys.stdout` object, with a bit of default formatting. In fact, if you enjoy working harder than you must, you can also code print operations this way:

```
>>> import sys                   # Printing the hard way
>>> sys.stdout.write('hello world\n')
hello world
```

This code explicitly calls the `write` method of `sys.stdout`—an attribute preset when Python starts up to an open file object connected to the output stream. The `print` operation hides most of those details, providing a simple tool for simple printing tasks.

Manual stream redirection

So, why did I just show you the hard way to print? The `sys.stdout` print equivalent turns out to be the basis of a common technique in Python. In general, `print` and `sys.stdout` are directly related as follows. This statement:

```
print(X, Y)                                # Or, in 2.6: print X, Y
```

is equivalent to the longer:

```
import sys
sys.stdout.write(str(X) + ' ' + str(Y) + '\n')
```

which manually performs a string conversion with `str`, adds a separator and newline with `+`, and calls the output stream's `write` method. Which would you rather code? (He says, hoping to underscore the programmer-friendly nature of prints....)

Obviously, the long form isn't all that useful for printing by itself. However, it is useful to know that this is exactly what `print` operations do because it is possible to *reassign* `sys.stdout` to something different from the standard output stream. In other words, this equivalence provides a way of making your `print` operations send their text to other places. For example:

```
import sys
sys.stdout = open('log.txt', 'a')          # Redirects prints to a file
...
print(x, y, x)                             # Shows up in log.txt
```

Here, we reset `sys.stdout` to a manually opened file named `log.txt`, located in the script's working directory and opened in append mode (so we add to its current content). After the reset, every `print` operation anywhere in the program will write its text to the end of the file `log.txt` instead of to the original output stream. The `print` operations are happy to keep calling `sys.stdout`'s `write` method, no matter what `sys.stdout` happens to refer to. Because there is just one `sys` module in your process, assigning `sys.stdout` this way will redirect every `print` anywhere in your program.

In fact, as this chapter's upcoming sidebar about `print` and `stdout` will explain, you can even reset `sys.stdout` to an object that isn't a file at all, as long as it has the expected interface: a method named `write` to receive the printed text string argument. When that object is a *class*, printed text can be routed and processed arbitrarily per a `write` method you code yourself.

This trick of resetting the output stream is primarily useful for programs originally coded with `print` statements. If you know that output should go to a file to begin with, you can always call file write methods instead. To redirect the output of a `print`-based

program, though, resetting `sys.stdout` provides a convenient alternative to changing every `print` statement or using system shell-based redirection syntax.

Automatic stream redirection

This technique of redirecting printed text by assigning `sys.stdout` is commonly used in practice. One potential problem with the last section's code, though, is that there is no direct way to restore the original output stream should you need to switch back after printing to a file. Because `sys.stdout` is just a normal file object, you can always save it and restore it if needed: #

```
C:\misc> c:\python30\python
>>> import sys
>>> temp = sys.stdout                # Save for restoring later
>>> sys.stdout = open('log.txt', 'a') # Redirect prints to a file
>>> print('spam')                   # Prints go to file, not here
>>> print(1, 2, 3)
>>> sys.stdout.close()              # Flush output to disk
>>> sys.stdout = temp                # Restore original stream

>>> print('back here')               # Prints show up here again
back here
>>> print(open('log.txt').read())     # Result of earlier prints
spam
1 2 3
```

As you can see, though, manual saving and restoring of the original output stream like this involves quite a bit of extra work. Because this crops up fairly often, a `print` extension is available to make it unnecessary.

In 3.0, the `file` keyword allows a single `print` call to send its text to a file's `write` method, without actually resetting `sys.stdout`. Because the redirection is temporary, normal `print` calls keep printing to the original output stream. In 2.6, a `print` statement that begins with a `>>` followed by an output file object (or other compatible object) has the same effect. For example, the following again sends printed text to a file named `log.txt`:

```
log = open('log.txt', 'a')           # 3.0
print(x, y, z, file=log)             # Print to a file-like object
print(a, b, c)                       # Print to original stdout

log = open('log.txt', 'a')           # 2.6
print >> log, x, y, z                 # Print to a file-like object
print a, b, c                        # Print to original stdout
```

These redirected forms of `print` are handy if you need to print to *both* files and the standard output stream in the same program. If you use these forms, however, be sure

#In both 2.6 and 3.0 you may also be able to use the `__stdout__` attribute in the `sys` module, which refers to the original value `sys.stdout` had at program startup time. You still need to restore `sys.stdout` to `sys.__stdout__` to go back to this original stream value, though. See the `sys` module documentation for more details.

to give them a file object (or an object that has the same `write` method as a file object), not a file's name string. Here is the technique in action:

```
C:\misc> c:\python30\python
>>> log = open('log.txt', 'w')
>>> print(1, 2, 3, file=log)           # 2.6: print >> log, 1, 2, 3
>>> print(4, 5, 6, file=log)
>>> log.close()
>>> print(7, 8, 9)                     # 2.6: print 7, 8, 9
7 8 9
>>> print(open('log.txt').read())
1 2 3
4 5 6
```

These extended forms of `print` are also commonly used to print error messages to the standard error stream, available to your script as the preopened file object `sys.stderr`. You can either use its file `write` methods and format the output manually, or print with redirection syntax:

```
>>> import sys
>>> sys.stderr.write(('Bad!' * 8) + '\n')
Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!

>>> print('Bad!' * 8, file=sys.stderr)   # 2.6: print >> sys.stderr, 'Bad' * 8
Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!
```

Now that you know all about print redirections, the equivalence between printing and file `write` methods should be fairly obvious. The following interaction prints both ways in 3.0, then redirects the output to an external file to verify that the same text is printed:

```
>>> X = 1; Y = 2
>>> print(X, Y)                        # Print: the easy way
1 2
>>> import sys
>>> sys.stdout.write(str(X) + ' ' + str(Y) + '\n')   # Print: the hard way
1 2
4
>>> print(X, Y, file=open('temp1', 'w'))           # Redirect text to file

>>> open('temp2', 'w').write(str(X) + ' ' + str(Y) + '\n') # Send to file manually
4
>>> print(open('temp1', 'rb').read())               # Binary mode for bytes
b'1 2\r\n'
>>> print(open('temp2', 'rb').read())
b'1 2\r\n'
```

As you can see, unless you happen to enjoy typing, print operations are usually the best option for displaying text. For another example of the equivalence between prints and file writes, watch for a 3.0 `print` function emulation example in [Chapter 18](#); it uses this code pattern to provide a general 3.0 `print` function equivalent for use in Python 2.6.

Version-Neutral Printing

Finally, if you cannot restrict your work to Python 3.0 but still want your prints to be compatible with 3.0, you have some options. For one, you can code 2.6 `print` statements and let 3.0's `2to3` conversion script translate them to 3.0 function calls automatically. See the Python 3.0 documentation for more details about this script; it attempts to translate 2.X code to run under 3.0.

Alternatively, you can code 3.0 `print` function calls in your 2.6 code, by enabling the function call variant with a statement like the following:

```
from __future__ import print_function
```

This statement changes 2.6 to support 3.0's `print` functions exactly. This way, you can use 3.0 print features and won't have to change your prints if you later migrate to 3.0.

Also keep in mind that simple prints, like those in the first row of [Table 11-5](#), work in *either* version of Python—because any expression may be enclosed in parentheses, we can always pretend to be calling a 3.0 `print` function in 2.6 by adding outer parentheses. The only downside to this is that it makes a tuple out of your printed objects if there are more than one—they will print with extra enclosing parentheses. In 3.0, for example, any number of objects may be listed in the call's parentheses:

```
C:\misc> c:\python30\python
>>> print('spam')                # 3.0 print function call syntax
spam
>>> print('spam', 'ham', 'eggs') # These are mutiple arguments
spam ham eggs
```

The first of these works the same in 2.6, but the second generates a tuple in the output:

```
C:\misc> c:\python26\python
>>> print('spam')                # 2.6 print statement, enclosing parens
spam
>>> print('spam', 'ham', 'eggs') # This is really a tuple object!
('spam', 'ham', 'eggs')
```

To be truly portable, you can format the print string as a single object, using the string formatting expression or method call, or other string tools that we studied in [Chapter 7](#):

```
>>> print('%s %s %s' % ('spam', 'ham', 'eggs'))
spam ham eggs
>>> print('{0} {1} {2}'.format('spam', 'ham', 'eggs'))
spam ham eggs
```

Of course, if you can use 3.0 exclusively you can forget such mappings entirely, but many Python programmers will at least encounter, if not write, 2.X code and systems for some time to come.



I use Python 3.0 `print` function calls throughout this book. I'll usually warn you that the results may have extra enclosing parentheses in 2.6 because multiple items are a tuple, but I sometimes don't, so please consider this note a blanket warning—if you see extra parentheses in your printed text in 2.6, either drop the parentheses in your `print` statements, recode your prints using the version-neutral scheme outlined here, or learn to love superfluous text.

Why You Will Care: `print` and `stdout`

The equivalence between the `print` operation and writing to `sys.stdout` is important. It makes it possible to reassign `sys.stdout` to any user-defined object that provides the same `write` method as files. Because the `print` statement just sends text to the `sys.stdout.write` method, you can capture printed text in your programs by assigning `sys.stdout` to an object whose `write` method processes the text in arbitrary ways.

For instance, you can send printed text to a GUI window, or tee it off to multiple destinations, by defining an object with a `write` method that does the required routing. You'll see an example of this trick when we study classes in [Part VI](#) of this book, but abstractly, it looks like this:

```
class FileFaker:
    def write(self, string):
        # Do something with printed text in string

import sys
sys.stdout = FileFaker()
print(someObjects)                # Sends to class write method
```

This works because `print` is what we will call in the next part of this book a *polymorphic* operation—it doesn't care what `sys.stdout` is, only that it has a method (i.e., interface) called `write`. This redirection to objects is made even simpler with the `file` keyword argument in 3.0 and the `>>` extended form of `print` in 2.6, because we don't need to reset `sys.stdout` explicitly—normal prints will still be routed to the `stdout` stream:

```
myobj = FileFaker()                # 3.0: Redirect to object for one print
print(someObjects, file=myobj)     # Does not reset sys.stdout

myobj = FileFaker()                # 2.6: same effect
print >> myobj, someObjects        # Does not reset sys.stdout
```

Python's built-in `input` function reads from the `sys.stdin` file, so you can intercept read requests in a similar way, using classes that implement file-like `read` methods instead. See the `input` and `while` loop example in [Chapter 10](#) for more background on this.

Notice that because printed text goes to the `stdout` stream, it's the way to print HTML in CGI scripts used on the Web. It also enables you to redirect Python script input and output at the operating system's shell command line, as usual:

```
python script.py < inputfile > outputfile  
python script.py | filterProgram
```

Python's print operation redirection tools are essentially pure-Python alternatives to these shell syntax forms.

Chapter Summary

In this chapter, we began our in-depth look at Python statements by exploring assignments, expressions, and print operations. Although these are generally simple to use, they have some alternative forms that, while optional, are often convenient in practice: augmented assignment statements and the redirection form of `print` operations, for example, allow us to avoid some manual coding work. Along the way, we also studied the syntax of variable names, stream redirection techniques, and a variety of common mistakes to avoid, such as assigning the result of an `append` method call back to a variable.

In the next chapter, we'll continue our statement tour by filling in details about the `if` statement, Python's main selection tool; there, we'll also revisit Python's syntax model in more depth and look at the behavior of Boolean expressions. Before we move on, though, the end-of-chapter quiz will test your knowledge of what you've learned here.

Test Your Knowledge: Quiz

1. Name three ways that you can assign three variables to the same value.
2. Why might you need to care when assigning three variables to a mutable object?
3. What's wrong with saying `L = L.sort()`?
4. How might you use the `print` operation to send text to an external file?

Test Your Knowledge: Answers

1. You can use multiple-target assignments (`A = B = C = 0`), sequence assignment (`A, B, C = 0, 0, 0`), or multiple assignment statements on three separate lines (`A = 0`, `B = 0`, and `C = 0`). With the latter technique, as introduced in [Chapter 10](#), you can also string the three separate statements together on the same line by separating them with semicolons (`A = 0; B = 0; C = 0`).

2. If you assign them this way:

```
A = B = C = []
```

all three names reference the same object, so changing it in-place from one (e.g., `A.append(99)`) will affect the others. This is true only for in-place changes to mutable objects like lists and dictionaries; for immutable objects such as numbers and strings, this issue is irrelevant.

3. The list `sort` method is like `append` in that it makes an in-place change to the subject list—it returns `None`, not the list it changes. The assignment back to `L` sets `L` to `None`, not to the sorted list. As we'll see later in this part of the book, a newer built-in function, `sorted`, sorts any sequence and returns a new list with the sorting result; because this is not an in-place change, its result can be meaningfully assigned to a name.
4. To print to a file for a single `print` operation, you can use 3.0's `print(X, file=F)` call form, use 2.6's extended `print >> file, X` statement form, or assign `sys.stdout` to a manually opened file before the `print` and restore the original after. You can also redirect all of a program's printed text to a file with special syntax in the system shell, but this is outside Python's scope.

if Tests and Syntax Rules

This chapter presents the Python `if` statement, which is the main statement used for selecting from alternative actions based on test results. Because this is our first in-depth look at *compound statements*—statements that embed other statements—we will also explore the general concepts behind the Python statement syntax model here in more detail than we did in the introduction in [Chapter 10](#). Because the `if` statement introduces the notion of tests, this chapter will also deal with Boolean expressions and fill in some details on truth tests in general.

if Statements

In simple terms, the Python `if` statement selects actions to perform. It's the primary selection tool in Python and represents much of the *logic* a Python program possesses. It's also our first compound statement. Like all compound Python statements, the `if` statement may contain other statements, including other `ifs`. In fact, Python lets you combine statements in a program sequentially (so that they execute one after another), and in an arbitrarily nested fashion (so that they execute only under certain conditions).

General Format

The Python `if` statement is typical of `if` statements in most procedural languages. It takes the form of an `if` test, followed by one or more optional `elif` (“else if”) tests and a final optional `else` block. The tests and the `else` part each have an associated block of nested statements, indented under a header line. When the `if` statement runs, Python executes the block of code associated with the first test that evaluates to true, or the `else` block if all tests prove false. The general form of an `if` statement looks like this:

```
if <test1>:                # if test
    <statements1>          # Associated block
elif <test2>:              # Optional elifs
    <statements2>
else:                      # Optional else
    <statements3>
```

Basic Examples

To demonstrate, let's look at a few simple examples of the `if` statement at work. All parts are optional, except the initial `if` test and its associated statements. Thus, in the simplest case, the other parts are omitted:

```
>>> if 1:
...     print('true')
...
true
```

Notice how the prompt changes to `...` for continuation lines when typing interactively in the basic interface used here; in IDLE, you'll simply drop down to an indented line instead (hit Backspace to back up). A blank line (which you can get by pressing Enter twice) terminates and runs the entire statement. Remember that `1` is Boolean `true`, so this statement's test always succeeds. To handle a false result, code the `else`:

```
>>> if not 1:
...     print('true')
... else:
...     print('false')
...
false
```

Multiway Branching

Now here's an example of a more complex `if` statement, with all its optional parts present:

```
>>> x = 'killer rabbit'
>>> if x == 'roger':
...     print("how's jessica?")
... elif x == 'bugs':
...     print("what's up doc?")
... else:
...     print('Run away! Run away!')
...
Run away! Run away!
```

This multiline statement extends from the `if` line through the `else` block. When it's run, Python executes the statements nested under the first test that is true, or the `else` part if all tests are false (in this example, they are). In practice, both the `elif` and `else` parts may be omitted, and there may be more than one statement nested in each section. Note that the words `if`, `elif`, and `else` are associated by the fact that they line up vertically, with the same indentation.

If you've used languages like C or Pascal, you might be interested to know that there is no `switch` or `case` statement in Python that selects an action based on a variable's value. Instead, *multiway branching* is coded either as a series of `if/elif` tests, as in the prior example, or by indexing dictionaries or searching lists. Because dictionaries and lists can be built at runtime, they're sometimes more flexible than hardcoded `if` logic:

```
>>> choice = 'ham'
>>> print({'spam': 1.25,          # A dictionary-based 'switch'
...       'ham': 1.99,          # Use has_key or get for default
...       'eggs': 0.99,
...       'bacon': 1.10}[choice])
1.99
```

Although it may take a few moments for this to sink in the first time you see it, this dictionary is a multiway branch—indexing on the key `choice` branches to one of a set of values, much like a `switch` in C. An almost equivalent but more verbose Python `if` statement might look like this:

```
>>> if choice == 'spam':
...     print(1.25)
... elif choice == 'ham':
...     print(1.99)
... elif choice == 'eggs':
...     print(0.99)
... elif choice == 'bacon':
...     print(1.10)
... else:
...     print('Bad choice')
...
1.99
```

Notice the `else` clause on the `if` here to handle the default case when no key matches. As we saw in [Chapter 8](#), dictionary defaults can be coded with `in` expressions, `get` method calls, or exception catching. All of the same techniques can be used here to code a default action in a dictionary-based multiway branch. Here's the `get` scheme at work with defaults:

```
>>> branch = {'spam': 1.25,
...           'ham': 1.99,
...           'eggs': 0.99}

>>> print(branch.get('spam', 'Bad choice'))
1.25
>>> print(branch.get('bacon', 'Bad choice'))
Bad choice
```

An `in` membership test in an `if` statement can have the same default effect:

```
>>> choice = 'bacon'
>>> if choice in branch:
...     print(branch[choice])
... else:
...     print('Bad choice')
...
Bad choice
```

Dictionaries are good for associating values with keys, but what about the more complicated actions you can code in the statement blocks associated with `if` statements? In [Part IV](#), you'll learn that dictionaries can also contain *functions* to represent more complex branch actions and implement general jump tables. Such functions appear as

dictionary values, may be coded as function names or `lambdas`, and are called by adding parentheses to trigger their actions; stay tuned for more on this topic in [Chapter 19](#).

Although dictionary-based multiway branching is useful in programs that deal with more dynamic data, most programmers will probably find that coding an `if` statement is the most straightforward way to perform multiway branching. As a rule of thumb in coding, when in doubt, err on the side of simplicity and readability; it's the "Pythonic" way.

Python Syntax Rules

I introduced Python's syntax model in [Chapter 10](#). Now that we're stepping up to larger statements like the `if`, this section reviews and expands on the syntax ideas introduced earlier. In general, Python has a simple, statement-based syntax. However, there are a few properties you need to know about:

- **Statements execute one after another, until you say otherwise.** Python normally runs statements in a file or nested block in order from first to last, but statements like `if` (and, as you'll see, loops) cause the interpreter to jump around in your code. Because Python's path through a program is called the *control flow*, statements such as `if` that affect it are often called *control-flow statements*.
- **Block and statement boundaries are detected automatically.** As we've seen, there are no braces or "begin/end" delimiters around blocks of code in Python; instead, Python uses the indentation of statements under a header to group the statements in a nested block. Similarly, Python statements are not normally terminated with semicolons; rather, the end of a line usually marks the end of the statement coded on that line.
- **Compound statements = header + ":" + indented statements.** All compound statements in Python follow the same pattern: a header line terminated with a colon, followed by one or more nested statements, usually indented under the header. The indented statements are called a *block* (or sometimes, a *suite*). In the `if` statement, the `elif` and `else` clauses are part of the `if`, but they are also header lines with nested blocks of their own.
- **Blank lines, spaces, and comments are usually ignored.** Blank lines are ignored in files (but not at the interactive prompt, when they terminate compound statements). Spaces inside statements and expressions are almost always ignored (except in string literals, and when used for indentation). Comments are always ignored: they start with a `#` character (not inside a string literal) and extend to the end of the current line.
- **Docstrings are ignored but are saved and displayed by tools.** Python supports an additional comment form called documentation strings (*docstrings* for short), which, unlike `#` comments, are retained at runtime for inspection. Docstrings are simply strings that show up at the top of program files and some statements. Python

ignores their contents, but they are automatically attached to objects at runtime and may be displayed with documentation tools. Docstrings are part of Python’s larger documentation strategy and are covered in the last chapter in this part of the book.

As you’ve seen, there are no variable type declarations in Python; this fact alone makes for a much simpler language syntax than what you may be used to. However, for most new users the lack of the braces and semicolons used to mark blocks and statements in many other languages seems to be the most novel syntactic feature of Python, so let’s explore what this means in more detail.

Block Delimiters: Indentation Rules

Python detects block boundaries automatically, by line *indentation*—that is, the empty space to the left of your code. All statements indented the same distance to the right belong to the same block of code. In other words, the statements within a block line up vertically, as in a column. The block ends when the end of the file or a lesser-indented line is encountered, and more deeply nested blocks are simply indented further to the right than the statements in the enclosing block.

For instance, [Figure 12-1](#) demonstrates the block structure of the following code:

```
x = 1
if x:
    y = 2
    if y:
        print('block2')
    print('block1')
print('block0')
```

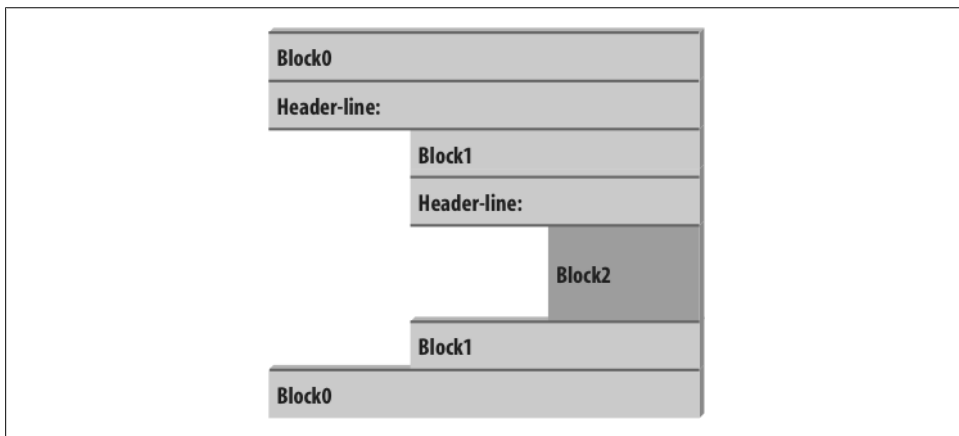


Figure 12-1. Nested blocks of code: a nested block starts with a statement indented further to the right and ends with either a statement that is indented less, or the end of the file.

This code contains three blocks: the first (the top-level code of the file) is not indented at all, the second (within the outer `if` statement) is indented four spaces, and the third (the `print` statement under the nested `if`) is indented eight spaces.

In general, top-level (unnested) code must start in column 1. Nested blocks can start in any column; indentation may consist of any number of spaces and tabs, as long as it's the same for all the statements in a given single block. That is, Python doesn't care *how* you indent your code; it only cares that it's done consistently. Four spaces or one tab per indentation level are common conventions, but there is no absolute standard in the Python world.

Indenting code is quite natural in practice. For example, the following (arguably silly) code snippet demonstrates common indentation errors in Python code:

```
x = 'SPAM'                                # Error: first line indented
if 'rubbery' in 'shrubbery':
    print(x * 8)
    x += 'NI'                              # Error: unexpected indentation
    if x.endswith('NI'):
        x *= 2
        print(x)                          # Error: inconsistent indentation
```

The properly indented version of this code looks like the following—even for an artificial example like this, proper indentation makes the code's intent much more apparent:

```
x = 'SPAM'
if 'rubbery' in 'shrubbery':
    print(x * 8)
    x += 'NI'
    if x.endswith('NI'):
        x *= 2
        print(x)                          # Prints "SPAMNISPAMNI"
```

It's important to know that the only major place in Python where whitespace matters is where it's used to the left of your code, for indentation; in most other contexts, space can be coded or not. However, indentation is really part of Python syntax, not just a stylistic suggestion: all the statements within any given single block must be indented to the same level, or Python reports a syntax error. This is intentional—because you don't need to explicitly mark the start and end of a nested block of code, some of the syntactic clutter found in other languages is unnecessary in Python.

As described in [Chapter 10](#), making indentation part of the syntax model also enforces consistency, a crucial component of readability in structured programming languages like Python. Python's syntax is sometimes described as “what you see is what you get”—the indentation of each line of code unambiguously tells readers what it is associated with. This uniform and consistent appearance makes Python code easier to maintain and reuse.

Indentation is more natural than the details might imply, and it makes your code reflect its logical structure. Consistently indented code always satisfies Python’s rules. Moreover, most text editors (including IDLE) make it easy to follow Python’s indentation model by automatically indenting code as you type it.

Avoid mixing tabs and spaces: New error checking in 3.0

One rule of thumb: although you can use spaces or tabs to indent, it’s usually not a good idea to mix the two within a block—use one or the other. Technically, tabs count for enough spaces to move the current column number up to a multiple of 8, and your code will work if you mix tabs and spaces consistently. However, such code can be difficult to change. Worse, mixing tabs and spaces makes your code difficult to read—tabs may look very different in the next programmer’s editor than they do in yours.

In fact, Python 3.0 now issues an error, for these very reasons, when a script mixes tabs and spaces for indentation inconsistently within a block (that is, in a way that makes it dependent on a tab’s equivalent in spaces). Python 2.6 allows such scripts to run, but it has a `-t` command-line flag that will warn you about inconsistent tab usage and a `-tt` flag that will issue errors for such code (you can use these switches in a command line like `python -t main.py` in a system shell window). Python 3.0’s error case is equivalent to 2.6’s `-tt` switch.

Statement Delimiters: Lines and Continuations

A statement in Python normally ends at the end of the line on which it appears. When a statement is too long to fit on a single line, though, a few special rules may be used to make it span multiple lines:

- **Statements may span multiple lines if you’re continuing an open syntactic pair.** Python lets you continue typing a statement on the next line if you’re coding something enclosed in a `()`, `{}`, or `[]` pair. For instance, expressions in parentheses and dictionary and list literals can span any number of lines; your statement doesn’t end until the Python interpreter reaches the line on which you type the closing part of the pair (a `)`, `}`, or `]`). Continuation lines (lines 2 and beyond of the statement) can start at any indentation level you like, but you should try to make them align vertically for readability if possible. This open pairs rule also covers set and dictionary comprehensions in Python 3.0.
- **Statements may span multiple lines if they end in a backslash.** This is a somewhat outdated feature, but if a statement needs to span multiple lines, you can also add a backslash (a `\` not embedded in a string literal or comment) at the end of the prior line to indicate you’re continuing on the next line. Because you can also continue by adding parentheses around most constructs, backslashes are almost never used. This approach is error-prone: accidentally forgetting a `\` usually generates a syntax error and might even cause the next line to be silently mistaken to be a new statement, with unexpected results.

- **Special rules for string literals.** As we learned in [Chapter 7](#), triple-quoted string blocks are designed to span multiple lines normally. We also learned in [Chapter 7](#) that adjacent string literals are implicitly concatenated; when used in conjunction with the open pairs rule mentioned earlier, wrapping this construct in parentheses allows it to span multiple lines.
- **Other rules.** There are a few other points to mention with regard to statement delimiters. Although uncommon, you can terminate a statement with a semicolon—this convention is sometimes used to squeeze more than one simple (noncompound) statement onto a single line. Also, comments and blank lines can appear anywhere in a file; comments (which begin with a # character) terminate at the end of the line on which they appear.

A Few Special Cases

Here's what a continuation line looks like using the open syntactic pairs rule. Delimited constructs, such as lists in square brackets, can span across any number of lines:

```
L = ["Good",
     "Bad",
     "Ugly"]                # Open pairs may span lines
```

This also works for anything in parentheses (expressions, function arguments, function headers, tuples, and generator expressions), as well as anything in curly braces (dictionaries and, in 3.0, set literals and set and dictionary comprehensions). Some of these are tools we'll study in later chapters, but this rule naturally covers most constructs that span lines in practice.

If you like using backslashes to continue lines, you can, but it's not common practice in Python:

```
if a == b and c == d and \
    d == e and f == g:
    print('olde')           # Backslashes allow continuations...
```

Because any expression can be enclosed in parentheses, you can usually use the open pairs technique instead if you need your code to span multiple lines—simply wrap a part of your statement in parentheses:

```
if (a == b and c == d and
    d == e and e == f):
    print('new')            # But parentheses usually do too
```

In fact, backslashes are frowned on, because they're too easy to not notice and too easy to omit altogether. In the following, `x` is assigned `10` with the backslash, as intended; if the backslash is accidentally omitted, though, `x` is assigned `6` instead, and no error is reported (the `+4` is a valid expression statement by itself).

In a real program with a more complex assignment, this could be the source of a very nasty bug:

```
x = 1 + 2 + 3 \           # Omitting the \ makes this very different
+4
```

As another special case, Python allows you to write more than one noncompound statement (i.e., statements without nested statements) on the same line, separated by semicolons. Some coders use this form to save program file real estate, but it usually makes for more readable code if you stick to one statement per line for most of your work:

```
x = 1; y = 2; print(x)    # More than one simple statement
```

As we learned in [Chapter 7](#), triple-quoted string literals span lines too. In addition, if two string literals appear next to each other, they are concatenated as if a + had been added between them—when used in conjunction with the open pairs rule, wrapping in parentheses allows this form to span multiple lines. For example, the first of the following inserts newline characters at line breaks and assigns `S` to `'\naaaa\nbbbb\ncccc'`, and the second implicitly concatenates and assigns `S` to `'aaaabbbbcccc'`; comments are ignored in the second form, but included in the string in the first:

```
S = """
aaaa
bbbb
cccc"""

S = ('aaaa'
    'bbbb'
    'cccc')           # Comments here are ignored
```

Finally, Python lets you move a compound statement’s body up to the header line, provided the body is just a simple (noncompound) statement. You’ll most often see this used for simple `if` statements with a single test and action:

```
if 1: print('hello')     # Simple statement on header line
```

You can combine some of these special cases to write code that is difficult to read, but I don’t recommend it; as a rule of thumb, try to keep each statement on a line of its own, and indent all but the simplest of blocks. Six months down the road, you’ll be happy you did.

* Frankly, it’s surprising that this wasn’t removed in Python 3.0, given some of its other changes! (See [Table P-2](#) of the Preface for a list of 3.0 removals; some seem fairly innocuous in comparison with the dangers inherent in backslash continuations.) Then again, this book’s goal is Python instruction, not populist outrage, so the best advice I can give is simply: don’t do this.